Project 2 — Choosing a sparse solver

David Cortes — CSC/DSE 589, Scientific Computing, Fall 2026 — November 10, 2026

How I measured. All the systems come from `problem.jl` ($n = m^d$, 5 or 7 nonzeros per row, SPD, with entries that span four orders of magnitude because of the permeability). Every number in this report was measured on the Innovator cluster, on a compute node with the same resources the grader uses (48 CPUs, 16 GB), with the scripts `run_direct.jl` and `run_iterative.jl` from my repo (github.com/david-neun/innovator-hpc, folder `projects/2`). They write one CSV row per run; the CSVs and the Slurm logs are in `results/`, and the figures below are made from those CSVs by `make_figures.jl` (Plots.jl), also run on the cluster, into `plot/`. I ran 2D from $m = 50$ to 1000 and 3D from $m = 10$ to 80. A time is always the total time of the operation, the way `test.jl` sees it: ordering + symbolic + numeric factorization + solve for Cholesky, and preconditioner setup + all the iterations + my residual check for CG. The first call of each method is thrown away so compilation is not counted. "Converged" always means the true relative residual $\|b - Ax\|_2 / \|b\|_2 \le$ `rtol`, which I compute myself after the solver returns. The direct solver is `cholesky` (CHOLMOD) with permutations from `SymRCM.jl`, `AMD.jl`, `Metis.jl`, or none. The iterative solver is `Krylov.cg` with BLAS set to one thread (and restored after), with the preconditioners `Diagonal(diag(A))`, `lldl(tril(A))` from `LimitedLDLFactorizations.jl` (incomplete LDL^T with no fill outside the pattern of A), and algebraic multigrid from `AlgebraicMultigrid.jl` (Ruge–Stüben and smoothed aggregation).

1. Orderings

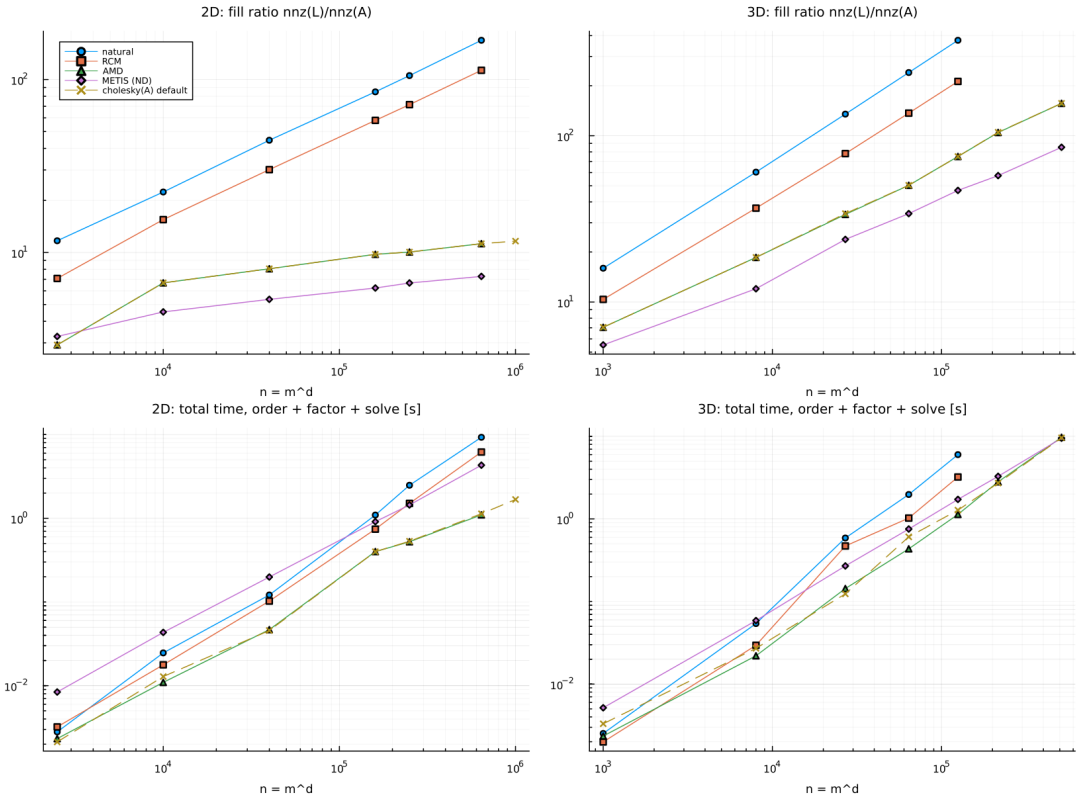


Figure 1: Sparse Cholesky under five orderings: fill ratio nnz(L)/nnz(A) and total time (order + factor + solve). The natural ordering in 3D stops at m = 50 because at m = 60 the job was killed for memory.

Which ordering does `cholesky(A)` use by default? AMD. At every size, the default factor has the same number of nonzeros as the factor I get when I pass the permutation from `AMD.jl` myself, to within 0.01% (2D $m = 500$: 12 548 562 vs 12 548 516; 3D $m = 80$: 555 677 728 vs 555 534 535). The permutation vectors are not identical because CHOLMOD post-orders the elimination tree, but that changes the order, not the fill. The default never matched METIS, even in 3D where METIS has 45% less fill.

Fill. The slopes of nnz(L) in the log–log plot match what we derived in class. Natural ordering: $n^{1.48}$ in 2D and $n^{1.67}$ in 3D; the band bound is nnz(L) $\approx n\beta$ with $\beta = m$ in 2D and m^2 in 3D, which is $n^{3/2}$ and $n^{5/3}$. RCM has the

same slopes with a smaller constant (fill 113 vs 169 at 2D $m = 800$; 212 vs 375 at 3D $m = 50$). On a grid the natural numbering already has a nearly minimal bandwidth, so RCM can only shave the profile. AMD and METIS change the exponent: $n^{1.2}$ in 2D (theory says $n \log n$) and $n^{1.5}$ in 3D (theory says $n^{4/3}$; at these sizes it is not asymptotic yet). The important data point is memory. At 3D $m = 80$, AMD's factor has $5.6 \cdot 10^8$ nonzeros (4.4 GB) and METIS's $3.0 \cdot 10^8$. **The natural ordering was killed by the 16 GB limit at $m = 60$, 70 and 80** (`oom_kill` in the Slurm log). The band bound predicts 6, 13 and 26 GB for L alone, and CHOLMOD needs workspace on top of that. So yes, the answer depends on d: in 2D the natural ordering still runs at $m = 800$, it is just 8× slower than AMD; in 3D it does not fit in memory at the grading size.

Is least fill always fastest? No. METIS has the least fill everywhere, and the numeric factorization by itself is the fastest with METIS (3D $m = 80$: 5.2 s vs 8.8 s with AMD; 2D $m = 500$: 0.35 s vs 0.43 s). But computing the METIS ordering costs 4.1 s at 3D $m = 80$ and 1.05 s at 2D $m = 500$, which eats the gain (3D: 9.5 s METIS vs 9.7 s AMD in total) or more than eats it (2D: 1.44 s vs 0.52 s). For one solve the ordering is part of the bill, so AMD (the default) is the fastest end to end at every size in both dimensions. At 2D $m = 500$ the totals are: default 0.53 s, AMD 0.52 s, METIS 1.44 s, RCM 1.51 s, natural 2.48 s. METIS would pay off if the factorization were reused for many right-hand sides.

2. Preconditioners

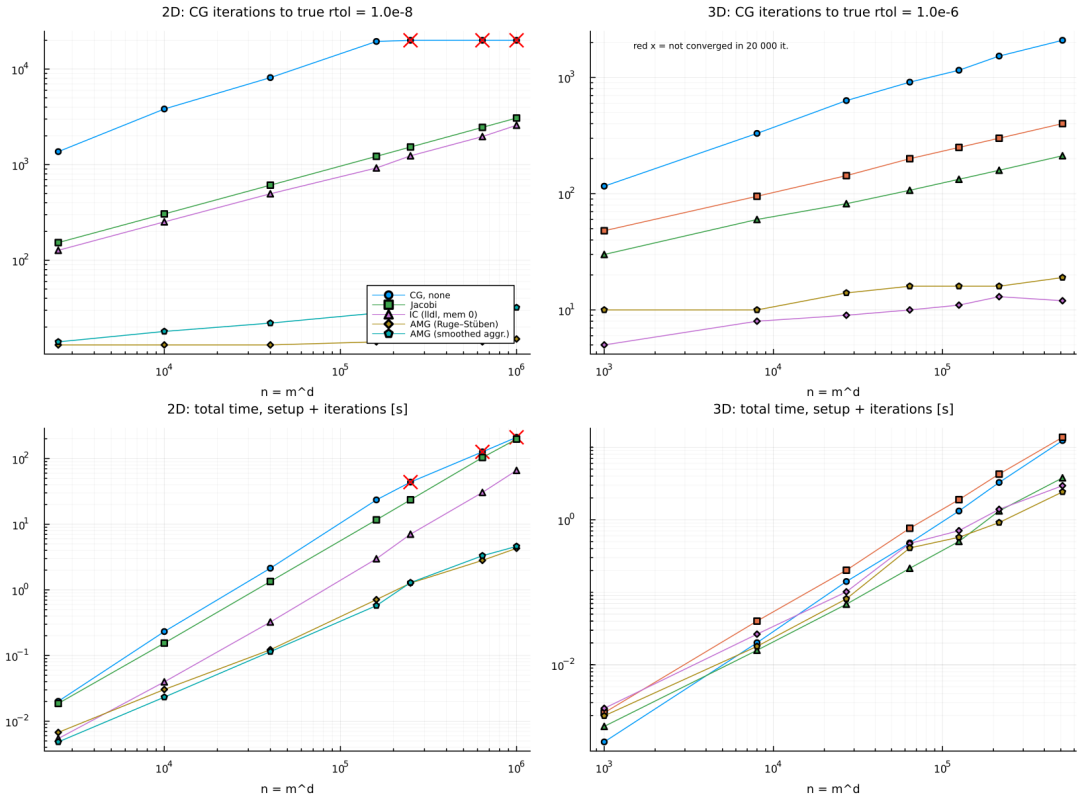


Figure 2: CG iterations and total time (setup + iterations) to reach the true residual at the official tolerances. Red × = not converged in 20 000 iterations.

How the iterations grow with n. The CG bound $k \approx \frac{\sqrt{\kappa}}{2} \ln \frac{2}{\varepsilon}$ with $\kappa \approx 0.4\, m^2$ says $k \propto m = n^{1/d}$. That is what I got. Jacobi: $m^{1.00}$ in 2D (153, 305, 610, 1222, 2455 iterations as m doubles) and $m^{1.03}$ in 3D (200, 250, 300, 401 at $m = 40, 50, 60, 80$). Incomplete Cholesky: $m^{0.99}$ and $m^{0.93}$, with 1.3 to 2 times fewer iterations than Jacobi. It improves κ by a constant factor, it does not change the exponent. Multigrid: 13 to 16 iterations at every 2D size up to $n = 10^6$ (slope $m^{0.05}$), and 5 to 12 in 3D up to $m = 80$. $\kappa(M^{-1}A)$ is basically independent of m, which is what the theory of multigrid says. Without preconditioner CG grows faster than m ($m^{1.26}$ and $m^{1.40}$) and needs 5 to 16 times more iterations than Jacobi; in 2D it did not converge in 20 000 iterations at $m = 500$, 800 or 1000. The difference between "none" and Jacobi is the permeability: the diagonal of A varies by 10^4, so $\kappa(A)$ carries that contrast on top of the grid's m^2, and dividing each row by its diagonal removes most of it for free.

Fewest iterations vs fastest — not the same thing. AMG always needs the fewest iterations. In 2D it is also the fastest ($m = 500$: Ruge–Stüben 1.27 s, smoothed aggregation 1.29 s, IC 7.0 s, Jacobi 23.6 s). In 3D the fastest method changes with the size: incomplete Cholesky up to $m = 50$ (0.21 s vs 0.41 s for AMG at $m = 40$), and from $m = 60$ on, smoothed-aggregation AMG (0.91 s vs 1.32 s IC at $m = 60$; at $m = 80$: smoothed aggregation 2.42 s, Ruge–Stüben 2.95 s, IC 3.78 s, IC with 20 extra entries per column 3.54 s, Jacobi 13.7 s, none 12.4 s). The reason is the cost structure: total time = setup + iterations × cost per iteration. An IC iteration is one sparse product plus two sparse triangular solves, and its setup is cheap, so it wins while the iteration count is small. One AMG V-cycle costs several sparse products over a hierarchy of grids and the setup is big (0.9 s for smoothed aggregation and 1.5 s for Ruge–Stüben at $m = 80$), but the iteration count does not grow, so AMG wins once m is large enough that IC's $O(m)$ iterations cost more than AMG's setup. Smoothed aggregation needs more iterations than Ruge–Stüben (19 vs 12 at $m = 80$) but its setup is 40% cheaper, and at this size the setup is what matters. The thread warning from the handout did not show up with this package: IC and AMG times were identical within 1% with 1 and 24 BLAS threads (`tune_blas_hpc.csv`), so the one-thread setting in `solve3d` is a precaution and not a measured gain.

The residual trap. Every preconditioned run in my CSVs has `restarts = 1`. `Krylov.cg` stops on the preconditioned residual $\|r\|_{M^{-1}}$ relative to its own starting value, and in every single case the true residual was still above `rtol` when it returned. My solvers recompute $\|b - Ax\|_2/\|b\|_2$ and warm-start CG from the returned x until it passes. Without this check, every preconditioned solve would have failed the grader's test while the package was reporting success.

3. Direct versus iterative

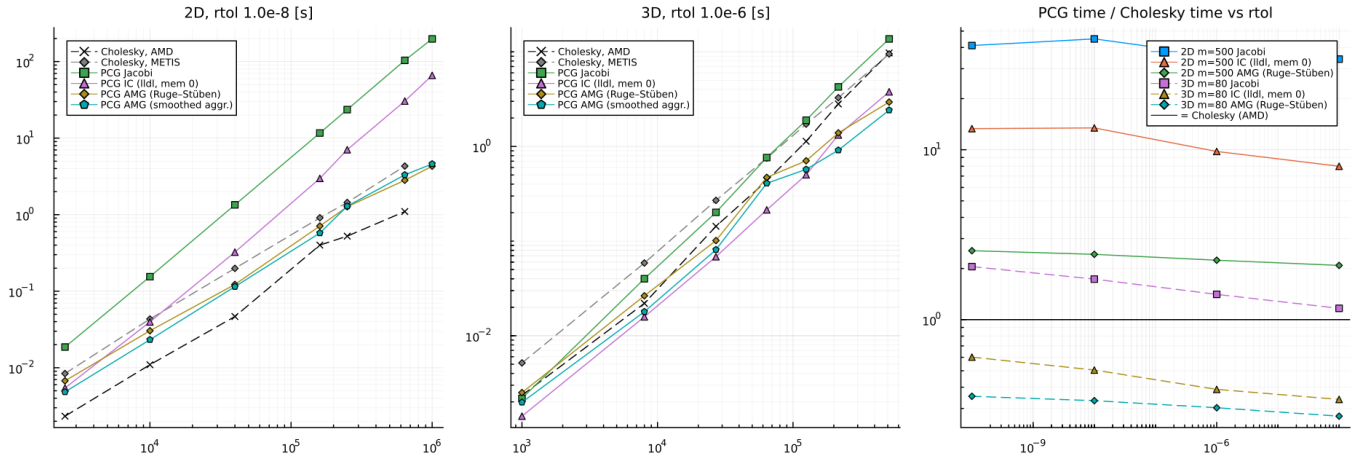


Figure 3: Left: 2D at rtol 1e-8. Middle: 3D at rtol 1e-6. Right: time(PCG)/time(Cholesky with AMD) as rtol changes at the official sizes; the direct time does not depend on rtol.

2D: there is no crossover. The best iterative method (AMG-PCG) is 1.8 to 3.2 times slower than the default Cholesky at every size from $n = 2\,500$ to 10^6 (1.27 s vs 0.53 s at $m = 500$; 4.30 s vs 1.68 s at $m = 1000$), and the ratio does not shrink, because both scale almost the same: $n^{1.13}$ for Cholesky and $n^{1.10}$ for AMG. In theory a 2D fill-reducing Cholesky costs $O(n^{3/2})$ flops but it runs them on dense supernodes at BLAS-3 speed, while a memory-bound PCG costs $O(n)$ per iteration with a large constant; at these sizes the constant wins and the exponents are too close for a crossover to appear. IC and Jacobi have worse exponents ($n^{1.58}$ and $n^{1.55}$, which is the $\sqrt{\kappa}$ growth of the iterations on top of the $O(n)$ cost per iteration) and fall further behind as n grows.

3D: the iterative method wins almost from the start, and the margin grows. IC-PCG is faster than AMD Cholesky at every size I ran: the ratio is 0.43 at $n = 1\,000$, 0.35 at 64 000 and 0.39 at 512 000. Smoothed-aggregation AMG crosses over around $m = 40$–50 and is at 0.25 of the direct time at $m = 80$ (2.42 s vs 9.74 s); Ruge–Stüben AMG at 0.30. Jacobi only wins at $m = 10$ and after that it is 1.3 to 1.6 times slower than the direct solver. The exponents explain it: Cholesky's total time grows like $n^{1.30}$ in 3D (fill $n^{1.5}$, work heading to the n^2 of 3D nested dissection), IC-PCG like $n^{1.27}$ with a much smaller constant, and AMG-PCG like $n^{1.17}$. The direct method also needs 4.4 GB for L at $m = 80$, against 60 MB for A.

Effect of `rtol`. The direct solve does not care about `rtol`: it always gives a residual around 10^{-11} to 10^{-15} and takes the same time. PCG's cost grows like $\ln(1/\texttt{rtol})$, as the CG bound says. At 3D $m = 80$, IC needs 165, 212, 273 and 335 iterations for $\texttt{rtol} = 10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}$ (about 55 more iterations per factor 100, linear in the logarithm), and

its time ratio to Cholesky goes from 0.34 to 0.60. AMG goes only from 0.27 to 0.35 because 9 to 17 iterations barely change its total. **Tightening `rtol` moves the crossover toward the direct method, loosening it moves it toward PCG.** At the grading sizes the margin is big enough that the decision does not flip for any tolerance between 10^{-4} and 10^{-10}, in either dimension (in 2D, AMG stays 2.1 to 2.6 times slower than Cholesky over the whole range).

4. My solvers

`solve2d` calls `cholesky(A)` and then `F \ b`. The default AMD ordering was the fastest end-to-end option at every 2D size (Fig. 1); METIS loses because the ordering costs more than it saves, and every PCG variant is at least 1.8 times slower (Fig. 3). A direct solve also meets any `rtol` without extra work. On the grading node, `test.jl -d 2 -m 500 --rtol 1e-8` gives **0.315 s** (baseline 4.5 s, roofline 0.5 s).

`solve3d` builds a smoothed-aggregation AMG hierarchy, uses one V-cycle as the preconditioner of `Krylov.cg`, and then checks the true residual and warm-starts if needed. BLAS is restricted to one thread during the solve and restored in a `finally` block. Two refinements came from a small tuning run at the grading size (`tune3d.jl`, `tune3d_b.jl`, results in `results/tune3d*_hpc.csv`). First, I ask Krylov for a tolerance 20 times tighter than `rtol`: its own stopping test uses the preconditioned residual and was always stopping too early, which cost a second CG call with new allocations; with the tighter inner tolerance the true residual passes on the first try (17 iterations, no restart, instead of 19 + a restart). Second, the smoother: one forward Gauss–Seidel sweep before the coarse correction and one backward sweep after, instead of the package's symmetric sweeps on both sides. The V-cycle stays symmetric (so M stays SPD, which CG needs) but it costs about half the smoothing work; it needs 23 iterations instead of 17, and it is still faster. The two changes together took the solve from 2.42 s to 1.87 s in the tuning runs. On the grading node, `test.jl -d 3 -m 80 --rtol 1e-6` gives **1.900 s** with a residual of $8.9 \cdot 10^{-7}$ (baseline 3.5 s, roofline 2.0 s). The Ruge–Stüben and incomplete-Cholesky variants are kept in the file behind one constant.

What surprised me. I wrote the first version of `solve3d` with incomplete Cholesky, because it was the fastest 3D method at every size I had tried up to $m = 50$. At the grading size it did not even beat the 3.5 s baseline (3.78 s), and multigrid did. The iteration counts were the ones I expected; what I had not appreciated is how fast IC's $O(m)$ iterations catch up with AMG's fixed setup once n is half a million. The crossover between two methods is not a property of the algorithms alone, it depends on the size and on the machine, and the only way to know is to measure at the size that will be graded.